Off-Policy GRPO for Online Speculative Decoding

MSAI-635 Interim Report

Gaurav Prakash UID: 122250130
Sanjana Kommareddy UID: 122099364
Simran Kharbanda UID: 122283671

1 Introduction and Scope

Speculative decoding lets a small *draft* model propose K tokens that a large *target* model verifies in one forward pass; verification keeps the longest correct prefix plus one target token, so the output is exactly the target's. The speed-up depends almost entirely on the mean acceptance length $\bar{\tau}$ (accepted draft tokens per verification step). Our proposal argued, following Aurora [1], that drafter training should be *online*, and proposed replacing Aurora's forward-KL distillation loss with an off-policy GRPO objective that rewards accepted length directly.

Change of plan since the proposal. The full Aurora stack needs about six 80 GB GPUs for Qwen3-8B (SGLang at TP=4 plus two FSDP training GPUs) and is CUDA-only (SGLang, Mooncake, Ray, FSDP2, FlexAttention). Before changing the objective we need a working, measurable forward-KL baseline that we fully control. We therefore split the project into two phases:

1. **Base-paper reproduction (in progress):** Aurora's online EAGLE-3 training, scaled down to Qwen3-0.6B on a single Apple M5 Pro laptop (24 GB unified memory, PyTorch MPS backend). We keep Aurora's algorithm and reuse its model, loss and data code, and replace only the serving and orchestration infrastructure. This phase is organised as nine sub-tasks (BP-01 to BP-09), each with its own issue, branch and pull request.
2. **GRPO extension (next):** the off-policy GRPO objective from the proposal, built on the reproduction. The reproduction already logs the quantities GRPO needs: per-round accepted length (the reward) and the draft version that produced each guess (the staleness Δ).

2 Progress So Far

Five of the nine base-paper sub-tasks are complete and merged. Each has a concrete "done when" criterion, summarised in Table 1.

Data (BP-02). Aurora's mixed-domain training data is private, so we built public replacements: 1000 conversations each from ShareGPT (chat), CodeAlpaca-20k (code) and GSM8K (math), filtered to start with a user turn and to fit in 1024 tokens. From these pools we emit three request streams: *mixed* (3000 requests, shuffled), *ordered* (3000, grouped by domain) and *domain-shift* (1200: 600 chat, then 300 code, then 300 math). The draft predicts only a reduced 24K-token vocabulary (out of Qwen3's 152K), built from assistant-token frequencies on the mixed stream with Aurora's `generate_vocab_mapping`.

Table 1: Base-paper sub-tasks and their status.

ID	Work	Done-when criterion	Status
BP-01	Mac environment, device layer (CUDA → MPS → CPU), Aurora installed without its CUDA-only dependencies	Aurora's draft and EAGLE-3 modules import; its parser and vocab tests pass	Done
BP-02	Public datasets (chat, code, math), three request streams, 24K draft-vocab mapping	Streams and cached mapping on disk	Done
BP-03	Qwen3-0.6B target with auxiliary hidden-state capture; EAGLE-3 draft and trainer step on MPS	Draft overfits one batch in 200 steps; time and memory measured	Done
BP-04	Offline static EAGLE-3 baseline trained on held-in chat data	$\bar{\tau}$ clearly > 0 on held-out chat	Done ($\bar{\tau} = 0.313$)
BP-05	Mini speculative-decoding server: draft, verify, accept, KV cache, logging, training samples, weight hot-swap	Greedy output token-identical to `generate` on 50 prompts	Done (50/50, 100/100)
BP-06	Online loop: sample pool, trainer, versioned weight sync (sync and async)	$\bar{\tau}$ rises over 300 day-0 requests	Next
BP-07	Metrics: rolling $\bar{\tau}$, α_i, speed-up, staleness, plots	Plots from logs; lossless regression test	Planned
BP-08	Experiments E1–E4 (Section 5)	Results table and figures	Planned
BP-09	Optional CUDA scale check (Qwen3-1.7B/4B)	Same code runs on CUDA	Optional

Models (BP-03). The target is Qwen3-0.6B (596M parameters), with auxiliary hidden states captured from layers 1, 13 and 24 via forward hooks. The EAGLE-3 draft is a single Llama decoder layer (hidden size 1024) fed by a linear fusion of the three auxiliary states (3072→1024) and the token embedding. It has 42.4M trainable parameters (24.6M of them in its own 24K-row output head) and a frozen 155.6M-parameter embedding copied from the target. Training uses Aurora's `Eagle3Model` with a three-step test-time-training (TTT) unroll and loss $\sum_i 0.8^i \mathcal{L}_i$, where $\mathcal{L}_i$ is the cross-entropy against the target's soft distribution at unroll step i.

Static baseline (BP-04). No public EAGLE-3 draft exists for Qwen3-0.6B, so we trained one offline. We dump the target's hidden states once for the chat pool, holding out the last 100 conversations, and train on the other 900: AdamW at learning rate 10^{-4}, warmup plus cosine schedule, effective batch 8, fp32 weights, gradient checkpointing, 8 epochs (896 steps, 19.4 minutes on MPS). This checkpoint is both the *static* baseline and the initialisation for Aurora's *with-draft* mode.

Inference server (BP-05). This replaces SGLang with an HF-transformers loop at batch size 1. Each round the draft proposes $K = 3$ guesses; the target scores all of them in one pass using a `DynamicCache` (cropped on rejection). We implement both greedy acceptance (exact argmax match) and speculative-sampling acceptance ($\min(1, p/q)$ with residual resampling). Every request is logged as one JSON line with per-round accepted lengths and draft versions. The server also emits Aurora-format training samples (prompt plus generated tokens, auxiliary hidden states, loss mask), and new draft weights can be loaded at any time with the version tracked. A test suite of 42 tests covers the data, models, baseline and server, including losslessness and a statistical test that speculative sampling matches the target's distribution.

3 Notional Architecture

Figure 1 shows the system. The solid components are implemented. The dashed components (sample pool, online trainer, weight publication) form the BP-06 online loop, the next deliverable. The GRPO extension changes only the loss inside the trainer and adds tree drafting to produce groups of rollouts.

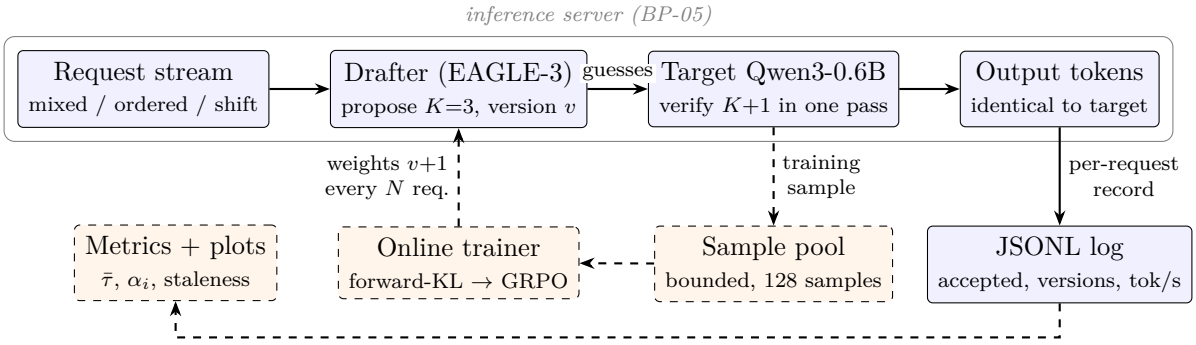


Figure 1: System architecture. Solid blue boxes are implemented (BP-02 to BP-05); dashed orange boxes are the online loop and metrics (BP-06, BP-07). The trainer's objective is forward KL in the reproduction and off-policy GRPO in the extension.

Table 2 maps each Aurora component to its laptop-scale replacement. The algorithmic pieces (draft architecture, TTT loss, vocab mapping, data parsing) are Aurora's own code, imported unchanged.

Table 2: Aurora components and our laptop-scale replacements.

Aurora (paper)	This reproduction
SGLang server with EAGLE-3 patch	HF `transformers` draft–verify loop on MPS
Mooncake transfer store	In-process bounded queue (same pool-size semantics)
Ray controller, placement groups	Plain Python loop (sync mode) or two threads (async mode)
FSDP2 `Eagle3Trainer`	Single-device trainer around Aurora's `Eagle3Model`
FlexAttention / FlashAttention, `torch.compile`	PyTorch SDPA, eager mode
Shared-filesystem weight sync	In-memory or on-disk `state_dict` with a version counter
Qwen3-8B / 4B target, 32K draft vocab	Qwen3-0.6B target, 24K draft vocab
$K = 5$, TTT length 5, ∼20K requests per run	$K = 3$, TTT length 3, 1–3K requests per run

4 Frameworks and Libraries

- **PyTorch 2.14** with the MPS backend (Apple GPU), with `PYTORCH_ENABLE_MPS_FALLBACK=1` and `TORCHDYNAMO_DISABLE=1`. Tests also run with the fallback disabled to prove that no operation silently falls back to the CPU.
- **Hugging Face `transformers` 4.57.1** (target model, `DynamicCache`, chat templates), `datasets` and `huggingface_hub` (ShareGPT, CodeAlpaca, GSM8K), `accelerate`, `safetensors`.
- **Aurora** [1] (`aurora-main/`, installed with `-no-deps`): EAGLE-3 draft model, `Eagle3Model` TTT wrapper, target-feature hooks, vocab mapping, conversation parser.
- **Tooling:** `uv` with Python 3.12, `pytest`, `matplotlib` and CSV for metrics (W&B offline is optional), Docker CPU image for non-Apple machines, Kaggle T4 notebooks for an independent

GPU check, GitHub issues and pull requests for the BP-01 to BP-09 workflow.

- **Optional scale-up (BP-09):** Aurora's original CUDA stack (SGLang, FSDP2, FlexAttention) on a Colab or cluster GPU with Qwen3-1.7B/4B, to check that the scaled-down results carry over.

5 Baselines and State of the Art

Lineage. Speculative decoding with lossless verification was introduced by Leviathan et al. [2] and Chen et al. [3]. Medusa [4] added parallel decoding heads to the target. The EAGLE family [5, 6, 7] drafts from the target's hidden states. EAGLE-3 fuses low-, mid- and high-level features and trains with a test-time-training unroll; it is the drafter used by Aurora and by us. Online Speculative Decoding [8] first proposed distilling the draft on live traffic. Aurora [1] makes this practical as a unified training–serving system with asynchronous weight hot-swap, and is the state of the art we reproduce. GRPO [9] is the critic-free RL objective our extension adapts.

Systems we compare. All are evaluated on the same streams, hardware and decoding settings:

1. **Target only:** plain autoregressive decoding, the speed-up reference.
2. **Static EAGLE-3:** the BP-04 draft, frozen. This is the "train offline, then deploy" pipeline Aurora argues against.
3. **Aurora, day-0:** online forward-KL training from a random draft.
4. **Aurora, with-draft:** online forward-KL training starting from the static draft.
5. **Ours:** off-policy GRPO with a forward-KL anchor, plus the ablations from the proposal: synchronous on-policy GRPO ($\Delta = 0$), group size G, KL weight β, clipping and truncation thresholds, and reward shaping.

Planned experiments (BP-08). **E1**: is day-0 online training better than target-only on the mixed stream? **E2**: static draft vs. with-draft online training under the chat→code→math shift (Aurora's main claim is faster recovery after a shift). **E3**: E2 on the ordered stream. **E4**: sensitivity to the sync interval $N \in \{5, 20, 50\}$, the learning rate, and sync vs. async training. The headline metric is $\bar{\tau}$, which does not depend on hardware. We also report per-position acceptance α_i, measured tokens/s, the estimated speed-up $(1+\bar{\tau})/(1+Kc)$ (where c is the measured draft-to-target cost ratio), the staleness distribution and the area under the rolling-$\bar{\tau}$ curve after each shift.

6 Evaluation Results (Prototype)

All numbers below are from the M5 Pro (MPS) with greedy decoding, $K = 3$, fp32, and up to 128 new tokens.

Correctness. Speculative output is token-identical to target-only `generate` on 50/50 stream prompts (18 chat, 17 code, 15 math) and on 100/100 held-out chat requests. This holds both with the KV cache and in the cache-free reference path.

Draft trainability (BP-03). On one fixed batch (4 sequences of 95 tokens) the draft overfits in 200 steps: loss falls from 9.87 to 0.851 against a target-entropy floor of 0.849, the remaining KL is 0.002, and accuracy is 0.99 / 0.99 / 0.98 at the three TTT positions. An AdamW step takes 98 ms and peak memory is 3.3 GB. On real 4×1025-token batches with the 24K vocab, peak memory is 9.9 GB in fp32, and 7.2 GB with bf16 and gradient checkpointing.

Static baseline (BP-04). Table 3 shows held-out accuracy (draft argmax equals target argmax, teacher-forced) during the 8-epoch run. The 2-epoch run gives the same numbers on a Kaggle T4 and on MPS (0.192 at position 1), which checks the two platforms against each other. Accuracy plateaus after epoch 6, as the cosine schedule decays the learning rate, and train accuracy is only ∼2 points above held-out, so the draft is not overfitting.

Table 3: BP-04 static draft on the 100 held-out chat conversations (random initialisation: loss $\ln 24000 = 10.09$).

Epoch	Step	Held-out loss (pos. 1 / 2 / 3)	Held-out accuracy (pos. 1 / 2 / 3)
1	112	6.46 / 6.72 / 6.85	0.183 / 0.138 / 0.119
2	224	5.75 / 5.96 / 6.11	0.245 / 0.209 / 0.191
4	448	5.19 / 5.43 / 5.58	0.292 / 0.254 / 0.232
6	672	5.00 / 5.25 / 5.41	0.308 / 0.268 / 0.245
8	**896**	**4.98 / 5.22 / 5.38**	**0.310 / 0.270 / 0.248**

Live serving (BP-05). Table 4 gives end-to-end results. The upper rows are sanity bounds: an untrained draft accepts nothing, while a draft overfitted to the exact answers reaches $\bar{\tau} = 2.6$ and 2.0× the target-only speed. This shows that the server can deliver a real speed-up when the draft guesses well. On held-out chat, the trained static draft reaches $\bar{\tau} = 0.313$, so the done-when criterion for BP-04 is met.

Table 4: Speculative serving on MPS. α_i is the share of rounds whose first i guesses were all accepted, so $\bar{\tau} = \alpha_1 + \alpha_2 + \alpha_3$.

Draft	Requests	$\bar{\tau}$	$\alpha_1/\alpha_2/\alpha_3$	tok/s (spec. vs. target)	Lossless
Untrained, KV cache	50 stream	0.00	–	33.4 vs. 59.9	50/50
Overfit (upper bound)	50 stream	2.60	–	122.3 vs. 59.9 (2.0×)	50/50
Static, 2 epochs	100 held-out	0.181	0.164 / 0.016 / 0.001	29.1 vs. 54.9 (0.53×)	100/100
Static, 8 epochs (baseline)	100 held-out	**0.313**	**0.242 / 0.058 / 0.013**	**32.1 vs. 54.8** (0.59×)	**100/100**

What the numbers say. (i) Acceptance collapses after the first guess ($\alpha_2 = 0.06$). A chain stops at the first miss, and guesses 2–3 are built on the draft's own hidden states. This is why teacher-forced accuracy (0.31 / 0.27 / 0.25) overstates live acceptance. (ii) Serving is still *slower* than the target alone: one round costs ∼41 ms, about 2.3 target-only steps of 18 ms, but yields only 1.32 tokens, so breaking even needs $\bar{\tau} \approx 1.3$. (iii) Every gain must therefore come from higher acceptance, which is what online training (BP-06) and the GRPO objective target, together with a cheaper round (a draft KV cache). Positions 2–3 are exactly where an acceptance-based reward differs most from forward KL, which weights all positions equally.

7 Issues Faced

Solved.

- **Compute and CUDA-only stack.** Full Aurora is out of reach, so we scaled down to Qwen3-0.6B on MPS and replaced SGLang, Mooncake, Ray and FSDP2 (Table 2), while reusing Aurora's algorithmic code unchanged.
- **No public draft and private data.** We trained our own EAGLE-3 draft (BP-04) on public datasets. In ShareGPT, 939 sampled threads started mid-conversation and 613 exceeded 1024 tokens; filters remove both.
- **Draft-vocab coverage.** A 16K draft vocab covered only 97.6% of assistant tokens, and tokens outside it can never be drafted. 24K raises coverage to 0.993 / 0.995 / 0.998 (chat / code / math) for 8.2M more parameters and ∼1 GB more peak memory, recovered by gradient checkpointing.
- **Memory on real sequences.** 4×1K-token batches needed 9.9 GB, beyond the 6–8 GB estimate, because of the draft logits across three TTT steps. We use batch 2 with gradient accumulation and gradient checkpointing (2.4 GB in training).
- **bf16 training underperforms.** With pure-bf16 AdamW at learning rate 10^{-4}, small updates round away (all held-out accuracies 2–3 points lower). Aurora avoids this with fp32 master weights under FSDP, so we train in fp32. A reload bug that silently cast fp32 checkpoints to bf16 was found and fixed.
- **Exactness in bf16.** Scoring four tokens in one pass rounds slightly differently from four single-token passes, which can flip near-tied argmaxes. Losslessness checks therefore run in fp32, where they pass 150/150.
- **Under-trained baseline.** The first 2-epoch draft gave $\bar{\tau} = 0.18$. Eight epochs raised it to 0.31 for 20 minutes of training.

Open.

- **No wall-clock speed-up yet** (0.59×), as analysed above. Mitigations: online training, a draft-side KV cache, and reporting the estimated speed-up next to the measured one, since MPS overheads differ from a production GPU server.
- **Train/test overlap in the streams.** The 600 chat requests of the domain-shift stream, and 900 of the 1000 chat requests in the mixed stream, were BP-04 training prompts. Before E2/E3 we must either rebuild the chat segments from held-out conversations or report results on the unseen segments only.
- **Single-device asynchrony.** Serving and training share one GPU, so "async" mode on the laptop competes for the same compute. Staleness will be real but smaller than in Aurora's two-server layout; E4 quantifies this.
- **GRPO prerequisites.** Groups of rollouts need tree drafting (top-$k > 1$), which is not yet implemented (chain drafting only), and the reduced $K = 3$ shortens the reward range compared with the paper's $K = 5$.
- **Sharing artifacts.** Checkpoints (170 MB) are not in git. We will publish the static draft as a GitHub release so all members start BP-06 from the same weights.

8 Next Steps

1. **BP-06, online loop:** a bounded sample pool fed by the server, the BP-03 trainer, and weight publication every N requests, in sync and async modes, from both day-0 and with-draft starts. Done when $\bar{\tau}$ rises over a 300-request day-0 run.
2. **BP-07, metrics:** rolling $\bar{\tau}$, α_i, speed-up, staleness histograms and loss curves from the JSONL logs, plus a losslessness regression test.
3. **BP-08, experiments E1–E4**, after fixing the stream overlap.
4. **GRPO extension:** tree drafting for groups, reward $R(a) = L(a) - \lambda\kappa(K - L(a))$ computed from the logged accepted lengths, the off-policy objective with truncated importance weights and the forward-KL anchor, and comparison against systems 2–4 above.

References

[1] J. Wang, F. Bie, J. Li, Z. Zhou, Z. Shao, Y. Wang, Y. Liu, Q. Wu, A. May, S. Yanamandra, Y. Zhang, C. Zhang, T. Dao, P. Liang, B. Athiwaratkun, S. L. Song, C. Xu, and X. Wu. When RL meets adaptive speculative training: A unified training–serving system. *arXiv:2602.06932*, 2026.

[2] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

[3] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. *arXiv:2302.01318*, 2023.

[4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *ICML*, 2024.

[5] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *ICML*, 2024.

[6] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *EMNLP*, 2024.

[7] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv:2503.01840*, 2025.

[8] X. Liu, L. Hu, P. Bailis, A. Cheung, Z. Deng, I. Stoica, and H. Zhang. Online speculative decoding. In *ICML*, 2024.

[9] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*, 2024.